



UNIVERSITÀ DELLA CALABRIA
DIPARTIMENTO DI INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Secondo progetto:

Mini-Alu a 8 e a 16 bit

Studenti:

Thomas Michienzi

Mat. 250780

Pasquale Morabito

Mat. 250633

Emanuele Nisticò

Mat. 251218

Professoressa del corso:

Stefania Perri

ANNO ACCADEMICO 2025/2026

Indice

1	Descrizione del circuito	2
2	Descrizione VHDL e istanziamento dei circuiti	3
3	Sintesi e simulazione post-sintesi	8
3.1	Simulazioni post-sintesi	9
3.2	Dati su utilizzazione	10
4	Implementazione e simulazione	
	post-implementazione	11
4.1	Simulazione post-implementazione	11
4.2	Dati su utilizzazione e potenza	12

1 Descrizione del circuito

L'obiettivo del progetto è creare un circuito aritmetico che ricevuti due ingressi ad n bit in complemento a due, produce il relativo output, pilotato da un segnale di comando (**cmd**). Esso ci permette di analizzare due casi:

- se **cmd** = '0', il circuito fornirà in output la somma dei valori rappresentati;
- se **cmd** = '1', il circuito fornirà in output la differenza dei valori rappresentati

Per realizzare questo circuito, consideriamo che l'operazione $A - B$ equivale alla somma $A + (-B)$ dove $-B$ è il complemento a 2 del valore di B , ovvero $-B = \overline{B} + '1'$. Per rendere il circuito così versatile mantenendolo compatto, utilizziamo il valore di comando sia come riporto iniziale per il circuito sommatore sia come bit di controllo di un multiplexer la cui uscita può essere B (se **cmd** = '0') o $\overline{B}$ (se **cmd** = '1'). Quindi inseriamo nel circuito un sommatore ad $n + 1$ bit con estensione del segno che prende in ingresso A (ad n bit), l'uscita del multiplexer (B o $\overline{B}$ ad n bit) e il segnale di comando come riporto iniziale. In questo modo riesco a calcolare immediatamente il complemento a 2 di B e il risultato del sommatore verrà interpretato come somma o come differenza in base al segnale di comando stesso. Il circuito appena descritto è rappresentato dalla seguente figura:

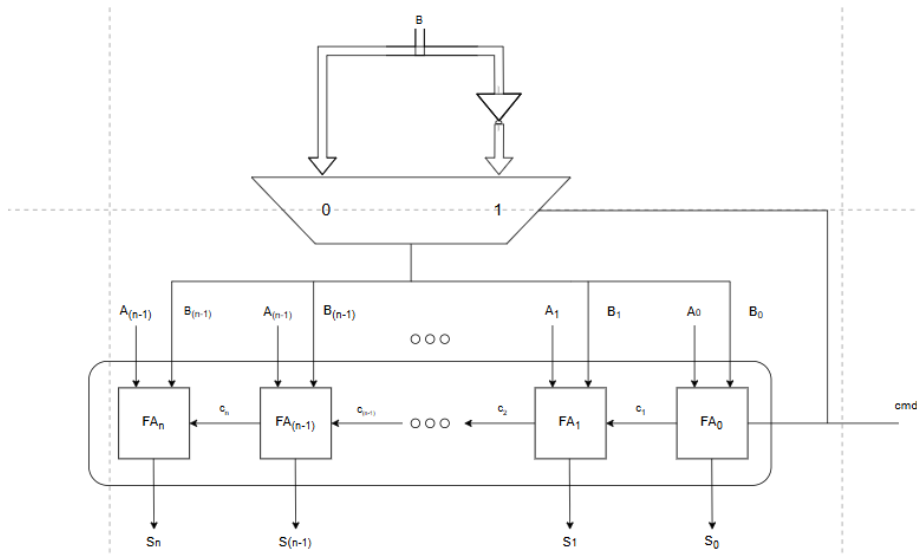


Figura 1: Schema del circuito

2 Descrizione VHDL e istanziazione dei circuiti

Traduciamo in VHDL il circuito descritto in precedenza:

Listing 1: File MiniALU.vhdl

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity MiniALU is
5     generic (n : integer := 4);
6     Port ( A : in STD_LOGIC_VECTOR (n - 1 downto 0);
7           B : in STD_LOGIC_VECTOR (n - 1 downto 0);
8           cmd : in STD_LOGIC;           -- segnale di comando
9           O : out STD_LOGIC_VECTOR (n downto 0));
10 end MiniALU;
11
12 architecture Behavioral of MiniALU is
13     signal EA, EB: STD_LOGIC_VECTOR (n downto 0);
14     signal p, g: STD_LOGIC_VECTOR (n downto 0);
15     signal c: STD_LOGIC_VECTOR (n + 1 downto 0);
16 begin
17     -- estensione in segno
18     EA <= A(n - 1) & A;
19     EB <= not(B(n - 1) & B) when cmd = '1' else
20         B(n - 1) & B when cmd = '0';
21
22     p <= EA xor EB;
23     g <= EA and EB;
24     c(0) <= cmd;
25     c(n + 1 downto 1) <= g or (p and c(n downto 0));
26
27     -- calcolo del risultato
28     O <= p xor c(n downto 0);
29 end Behavioral;
```

La descrizione fornita dal codice, tramite il costrutto *generic* rimane generica rispetto alla dimensione dei segnali interni al circuito. Istanziamo le versioni del circuito ad 8 bit e a 16 bit:

Listing 2: File MiniALU8b.vhdl

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity MiniALU8b is
5     Port( A : in STD_LOGIC_VECTOR (7 downto 0);
6           B : in STD_LOGIC_VECTOR (7 downto 0);
7           cmd: in STD_LOGIC;
8           O: out STD_LOGIC_VECTOR(8 downto 0));
9 end MiniALU8b;
```

```

10
11 architecture Behavioral of MiniALU8b is
12     component MiniALU
13         generic (n : integer);
14         Port ( A : in STD_LOGIC_VECTOR (n - 1 downto 0);
15               B : in STD_LOGIC_VECTOR (n - 1 downto 0);
16               cmd : in STD_LOGIC;
17               O : out STD_LOGIC_VECTOR (n downto 0));
18     end component;
19 begin
20     ALU: MiniALU generic map(8) port map(A, B, cmd, O);
21 end Behavioral;

```

Listing 3: File MiniALU16b.vhdl

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity MiniALU16b is
5     Port( A : in STD_LOGIC_VECTOR (15 downto 0);
6           B : in STD_LOGIC_VECTOR (15 downto 0);
7           cmd: in STD_LOGIC;
8           O : out STD_LOGIC_VECTOR(16 downto 0));
9 end MiniALU16b;
10
11 architecture Behavioral of MiniALU16b is
12     component MiniALU
13         generic (n : integer);
14         Port ( A : in STD_LOGIC_VECTOR (n - 1 downto 0);
15               B : in STD_LOGIC_VECTOR (n - 1 downto 0);
16               cmd : in STD_LOGIC;
17               O : out STD_LOGIC_VECTOR (n downto 0));
18     end component;
19 begin
20     ALU16b: MiniALU generic map(16) port map(A, B, cmd, O);
21 end Behavioral;

```

Tramite il software Vivado riusciamo a ricavare la rappresentazione logica dei due circuiti:

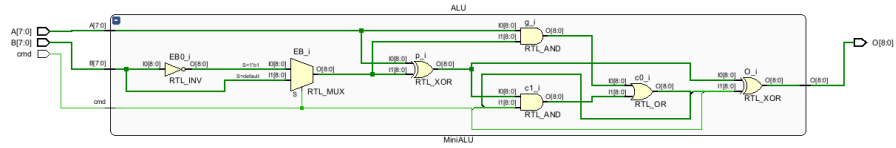


Figura 2: Rappresentazione logica del circuito a 8 bit

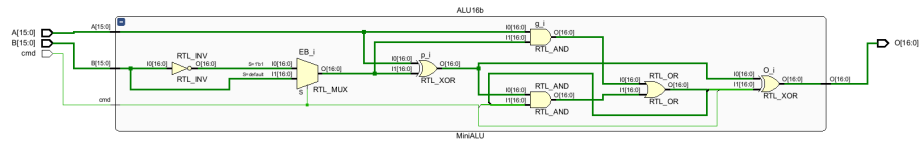


Figura 3: Rappresentazione logica del circuito a 16 bit

Per testare i due circuiti utilizziamo i seguenti test bench:

Listing 4: File TestBench8b.vhdl

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.std_logic_arith.ALL;
4
5  entity TestBench8b is
6  end TestBench8b;
7
8  architecture Behavioral of TestBench8b is
9      component MiniALU8b
10         Port( A: in STD_LOGIC_VECTOR (7 downto 0);
11              B: in STD_LOGIC_VECTOR (7 downto 0);
12              cmd: in STD_LOGIC;
13              O: out STD_LOGIC_VECTOR(8 downto 0));
14     end component;
15     signal A: STD_LOGIC_VECTOR (7 downto 0);
16     signal B: STD_LOGIC_VECTOR (7 downto 0);
17     signal cmd: STD_LOGIC;
18     signal O: STD_LOGIC_VECTOR(8 downto 0);
19 begin
20     circ: MiniALU8b port map(A, B, cmd, O);
21     process begin
22         -- Come sommatore
23         cmd<= '0';
24         for i in -128 to 127 loop

```

```

25         A <= conv_std_logic_vector(i, 8);
26         for j in -128 to 127 loop
27             B <= conv_std_logic_vector(j, 8);
28             wait for 10ns;
29         end loop;
30     end loop;
31     -- Come sottrattore
32     cmd <= '1';
33     for i in -128 to 127 loop
34         A <= conv_std_logic_vector(i, 8);
35         for j in -128 to 127 loop
36             B <= conv_std_logic_vector(j, 8);
37             wait for 10ns;
38         end loop;
39     end loop;
40 end process;
41 end Behavioral;

```

Listing 5: File TestBench16b.vhdl

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.std_logic_arith.ALL;
4
5  entity TestBench16b is
6  end TestBench16b;
7
8  architecture Behavioral of TestBench16b is
9      component MiniALU16b
10         Port( A: in STD_LOGIC_VECTOR (15 downto 0);
11              B: in STD_LOGIC_VECTOR (15 downto 0);
12              cmd: in STD_LOGIC;
13              O: out STD_LOGIC_VECTOR(16 downto 0));
14     end component;
15     signal A: STD_LOGIC_VECTOR (15 downto 0);
16     signal B: STD_LOGIC_VECTOR (15 downto 0);
17     signal cmd: STD_LOGIC;
18     signal O: STD_LOGIC_VECTOR(16 downto 0);
19 begin
20     circ: MiniALU16b port map(A, B, cmd, O);
21     process begin
22         -- 0 + 2*15 - 1
23         cmd <= '0';
24         A <= (others => '0');
25         B <= (others => '1');
26         B(15) <= '0';
27         wait for 20ns;
28
29         -- -1 + (-1)
30         A <= (others => '1');

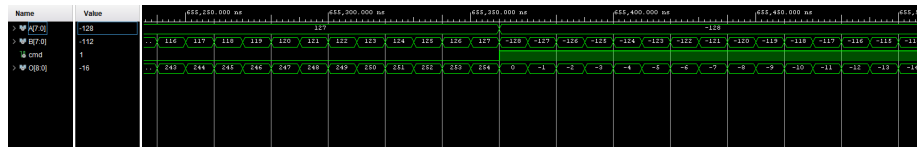
```

```

31     B <= (others => '1');
32     wait for 20ns;
33
34     -- 2**15 - 1 + (-2**15-1)
35     A(15) <= '0';
36     B <= (others => '0');
37     B(15) <= '1';
38     B(0) <= '1';
39     wait for 20ns;
40
41     -- 2**15 - 1 - (-2**15-1)
42     cmd <= '1';
43     wait for 20ns;
44
45     -- 0 - 1
46     A <= (others => '0');
47     B(15) <= '0';
48     wait for 20ns;
49 end process;
50 end Behavioral;

```

Simuliamo quindi il funzionamento logico-ideale del circuito. Per il circuito a 8 bit viene mostrato solamente il funzionamento per gli ingressi $A = 127$, $B = 116, 117, \dots, 126, 127$ con $\text{cmd} = 0$ e $A = -128$, $B = -128, -127, \dots, -114$, con $\text{cmd} = 1$.



3 Sintesi e simulazione post-sintesi

Svolta la descrizione del circuito attraverso il linguaggio VHDL e una prima simulazione logica, possiamo passare alla successiva fase di sintesi. In questo processo, attraverso il tool Vivado, andremo a tradurre la descrizione di alto livello in una rete di porte logiche implementabili sull' FPGA (Field Programmable Gate Array). Nel caso di questa tecnologia, le varie porte sono realizzate attraverso elementi configurabili, anche detti LUT (Look-Up Table), realizzati attraverso dei registri che contengono al loro interno delle tabelle di verità che, in fase di esecuzione, vengono acceduti tramite gli ingressi del blocco logico. Per i due sommatori dovremo seguire due fasi di sintesi distinte che nel nostro caso hanno portato al seguente schema per il sommatore ad 8bit

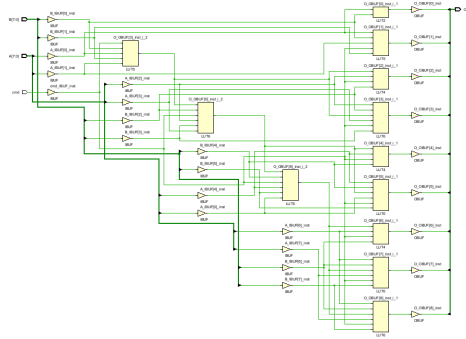


Figura 6: Schema dopo aver svolto la sintesi del circuito a 8 bit

ed il seguente schema per il 16 bit.

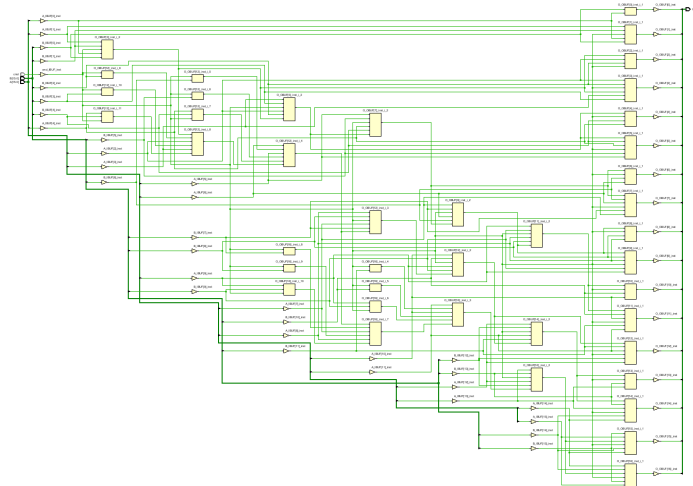


Figura 7: Schema dopo aver svolto la sintesi del circuito a 16 bit

3.1 Simulazioni post-sintesi

La fase di sintesi ci permette anche di introdurre nelle nostre simulazioni dei circuiti delle non idealità dovute alla logica implementata dalla tecnologia FPGA. In dettaglio, eseguendo una simulazione post-sintesi, riusciamo a visualizzare dei ritardi nella produzione dell'output dovuti all'elaborazione dei risultati delle LUT alla variazione dei segnali d'ingresso. È importante notare che questi ritardi sono stimati in quanto la sintesi simula solamente il funzionamento delle LUT dal punto di vista logico ma non considera ancora il circuito dal punto di vista fisico. Quest'analisi verrà svolta nella successiva fase di implementazione.

Anche per la simulazione post-sintesi utilizziamo i due testbench già realizzati. Per il circuito a 8 bit mostriamo, anche questa volta, il suo funzionamento per gli ingressi $A = 127$, $B = 124, 125, 126, 127$ con **cmd** = 0 e $A = -128$, $B = -128, -127, -126, -125$ con **cmd** = 1.

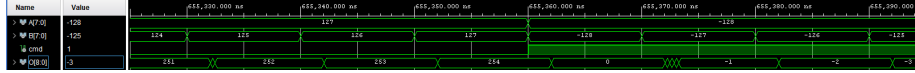


Figura 8: Simulazione temporale post-sintesi del circuito a 8 bit

Per il circuito a 16 bit mostriamo invece il suo funzionamento per tutti i casi di test.

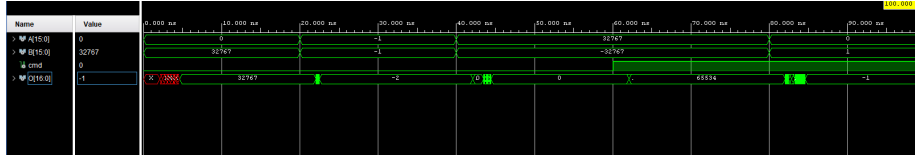


Figura 9: Simulazione temporale post-sintesi del circuito a 16 bit

Come possiamo notare dalle simulazioni, entrambi i circuiti restituiscono i segnali di output con un determinato ritardo rispetto a quando ricevono i segnali in ingresso. Considerando, ad esempio, per il circuito a 8 bit, la configurazione d'ingresso $A = 127$, $B = 126$, $cmd = 0$, notiamo che il risultato ($O = 253$) viene prodotto circa 2 ns dopo rispetto all'istante in cui lo stesso risultato veniva fornito nella simulazione logica (ovvero l'istante in cui sono stati forniti i segnali di input). Analogamente, considerando per la versione a 16 bit la configurazione d'ingresso $A = 0$, $B = 1$, $cmd = 1$, notiamo che si deve attendere un ritardo di circa 5 ns prima di ottenere il risultato corretto.

Facendo riferimento alle figure, notiamo che durante questi ritardi i segnali di output variano repentinamente partendo dalla configurazione del risultato pre-

cedente fino a stabilizzarsi poi al risultato corretto. Possiamo quindi dire che questi segnali devono mutare da uno stato iniziale stabilito, ovvero il risultato precedente. Il problema si pone durante il ritardo del primo calcolo del circuito, dove, non essendoci alcuno stato iniziale, non si può determinare lo stato dei segnali. Questo stato di indeterminazione viene convenzionalmente rappresentato con una 'X' (convenzione adottata dalla IEEE nel tipo *STD_LOGIC*).

Ovviamente, anche nel circuito ad 8 bit è presente questo comportamento, come mostra la seguente figura:

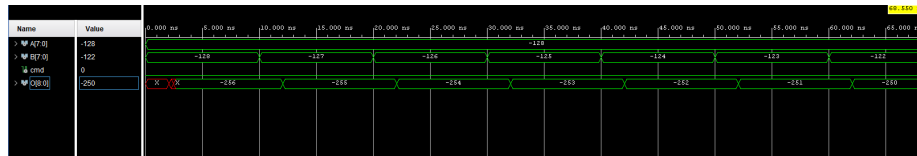


Figura 10: Simulazione temporale post-sintesi del circuito a 8 bit

3.2 Dati su utilizzazione

L'uso di una tecnologia più reale e di basso livello come l'FPGA ci fornisce il pretesto per iniziare a misurare i costi della nostra implementazione. Questi, nel caso della tecnologia FPGA, vengono espressi in termini di LUT utilizzate e di buffer di ingresso e di uscita presenti nel circuito e prendono il nome di costi di utilizzazione.

Il report fornito dal software Vivado ci fornisce i seguenti dati sui costi di utilizzazione delle due versioni del circuito:

Listing 6: Costi di utilizzazione circuito 8 bit

Ref	Name	Used	Functional Category
IBUF		17	IO
OBUF		9	IO
LUT6		6	LUT
LUT4		3	LUT
LUT5		2	LUT
LUT2		1	LUT

Listing 7: Costi di utilizzazione circuito 16 bit

Ref Name	Used	Functional Category
IBUF	33	IO
LUT6	21	LUT
OBUF	17	IO
LUT2	9	LUT
LUT4	6	LUT
LUT3	4	LUT
LUT5	2	LUT

Specifichiamo che nel report le LUT vengono divise in base al numero di ingressi che ammettono che è sempre un numero tra due e sei.

4 Implementazione e simulazione post-implementazione

Dopo aver svolto la fase di sintesi, possiamo passare alla fase di implementazione, in cui viene deciso dove mappare il circuito sintetizzato e, in seguito, come montarlo fisicamente sull'hardware dell'FPGA.

4.1 Simulazione post-implementazione

Dopo l'implementazione il circuito che stiamo simulando non viene più considerato dal solo punto di vista logico, ma anche da quello fisico. Questo fatto ci permette di considerare nuove non idealità fisiche introdotte dai percorsi di interconnessione sulla matrice del dispositivo FPGA. Quindi i ritardi del circuito non verranno più stimati ma calcolati realmente sullo schema fisico del dispositivo.

Mostriamo di seguito alcuni risultati ottenuti dalla simulazione fatta per la versione a 8 bit del circuito:

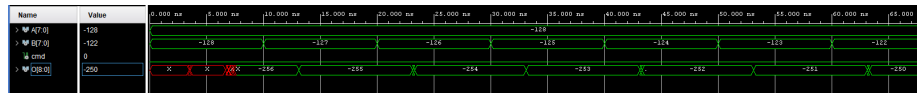


Figura 11: Simulazione temporale post-implementazione del circuito a 8 bit

Per la versione a 16 bit mostriamo, anche in questo caso, il funzionamento del circuito per tutti i casi di test considerati nel testbench:

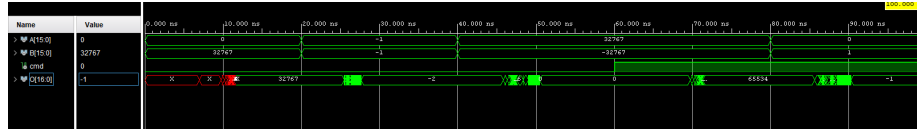


Figura 12: Simulazione temporale post-implementazione del circuito a 16 bit

Dalle figure notiamo che i ritardi con cui si hanno i segnali di uscita rispetto all'immissione dei segnali di ingresso sono mediamente aumentati. Notiamo, ad esempio, che il circuito ad 8 bit per fornire il risultato dell'operazione $-128 + (-127)$, ovvero $O = -255$, impiega circa $3ns$, mentre la versione a 16 bit, quando riceve in ingresso la configurazione $A = 0$, $B = 1$, $cmd = 1$, fornisce il risultato, ovvero $O = -1$ dopo circa $10ns$. Il comportamento del circuito durante questi ritardi somiglia a quello stimato nella simulazione post-sintesi, infatti, durante il ritardo del primo calcolo del circuito, i segnali di output sono indeterminati ('X').

4.2 Dati su utilizzazione e potenza

Avendo inserito il nostro circuito nel dispositivo riusciamo ad avere dei dati sull'utilizzazione di esso reali e non stimati come nella fase di sintesi. In genere, per circuiti di grandi dimensioni, i dati di utilizzazione ricavati in questa fase sono leggermente diversi rispetto a quelli stimati in precedenza. Nel nostro caso, invece, la stima fatta dal software nella fase di sintesi è corretta, infatti i dati ricavati sono identici a quelli precedenti, come mostrano le seguenti:

Listing 8: Costi di utilizzazione circuito 8 bit

Ref Name	Used	Functional Category
IBUF	17	IO
obuf	9	IO
LUT6	6	LUT
LUT4	3	LUT
LUT5	2	LUT
LUT2	1	LUT

Listing 9: Costi di utilizzazione circuito 16 bit

Ref	Name	Used	Functional Category
IBUF		33	I/O
LUT6		21	LUT
OBUF		17	I/O
LUT2		9	LUT
LUT4		6	LUT
LUT3		4	LUT
LUT5		2	LUT

Poiché stiamo considerando il circuito dal punto di vista fisico possiamo effettuare delle misure sulla potenza dissipata, in particolare sulla potenza statica, dinamica e quella sulle porte di I/O. Il software calcola i valori di potenza statica e dinamica tramite le seguenti formule:

$$P_{stat} = V_{DD} \cdot I$$

$$P_{dinamica} = f \cdot C_L \cdot V_{DD}^2$$

dove V_{DD} è la tensione di alimentazione del circuito, I è la corrente che passa nel circuito, f è la frequenza con cui varia la tensione nei transistori nel caso dinamico e C_L è la risultante delle capacità dei transistori del circuito. È importante notare che quando la tensione nel circuito rimane costante, non si hanno effetti capacitivi, infatti ha senso calcolare la potenza dinamica quando la tensione nei vari transistori varia nel tempo.

I valori forniti dal software sono:

Listing 10: Dati sulla potenza del circuito a 8 bit

Type	Power [W]	Percentage
statica	0.5	8%
dinamica	6.018	92%
I/O	5.739	95% della dinamica

Listing 11: Dati sulla potenza del circuito a 16 bit

Type	Power [W]	Percentage
statica	1.039	7%
dinamica	13.252	93%
I/O	11.430	86% della dinamica

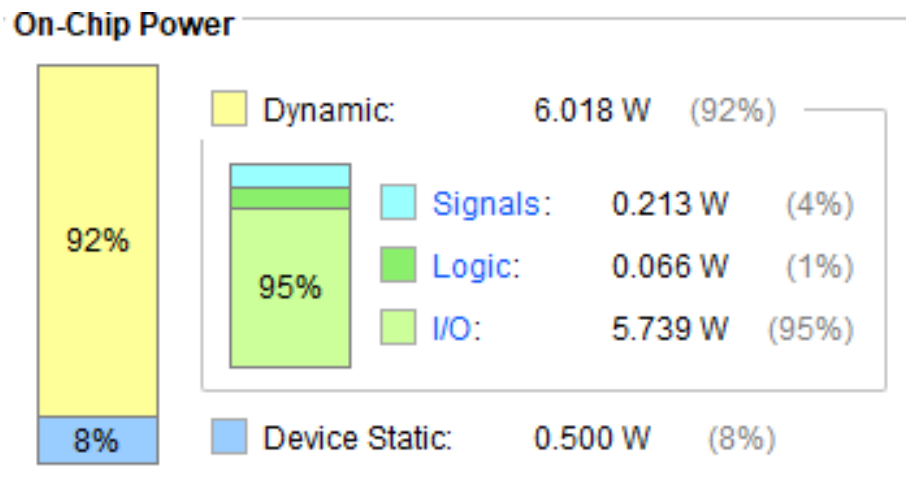


Figura 13: Report sulla potenza circuito a 8 bit

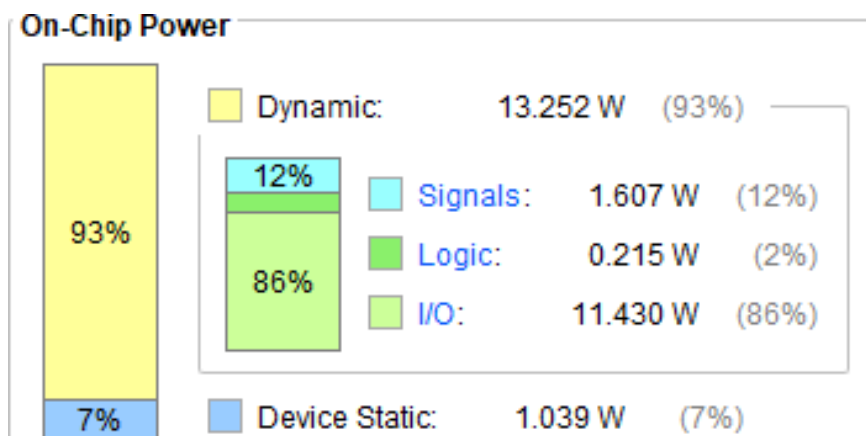


Figura 14: Report sulla potenza circuito a 16 bit

Si può notare come la potenza dinamica ottenuta è molto elevata, ciò accade perché non abbiamo esplicitamente indicato una frequenza di funzionamento del circuito e, di conseguenza, il tool impiega il massimo valore possibile. Questo scenario in una reale implementazione non sarebbe perseguibile dato che elevati valori di questo parametro si traducono in una elevata dissipazione di calore per effetto Joule.